Collision Detection and Obstacle Avoidance Planning

Preface: On the Raspberry Pi 5 and Jetson-nano, ROS runs in Docker, so the MoveIt2 experience is limited; it is recommended that Raspberry Pi 5 and Jetson-nano users run MoveIt2-related cases on a virtual machine. On the Orin board, ROS runs directly on the board, so Orin board users can run MoveIt2-related cases directly on the board, using the same commands as running on a virtual machine. This section will use running on a virtual machine as the example for demonstration.

1. Content Description

MoveIt2 not only provides powerful inverse kinematics solving capability, its more core advantage lies in its built-in environment collision detection engine (based on the FCL fast collision library) and sampling-based path planning algorithms (such as RRT*, PRM in OMPL, etc.).

This course introduces how to dynamically add 3D obstacles (such as pillars and cubes) into MoveIt2's planning scene (`PlanningScene`), and call the `/plan_kinematic_path` path planning service. While planning the robot arm to move from the `up` pose to the `down` pose, the algorithm automatically senses the obstacle boundaries in space and automatically searches for a collision-free motion trajectory that safely avoids the obstacles.

---

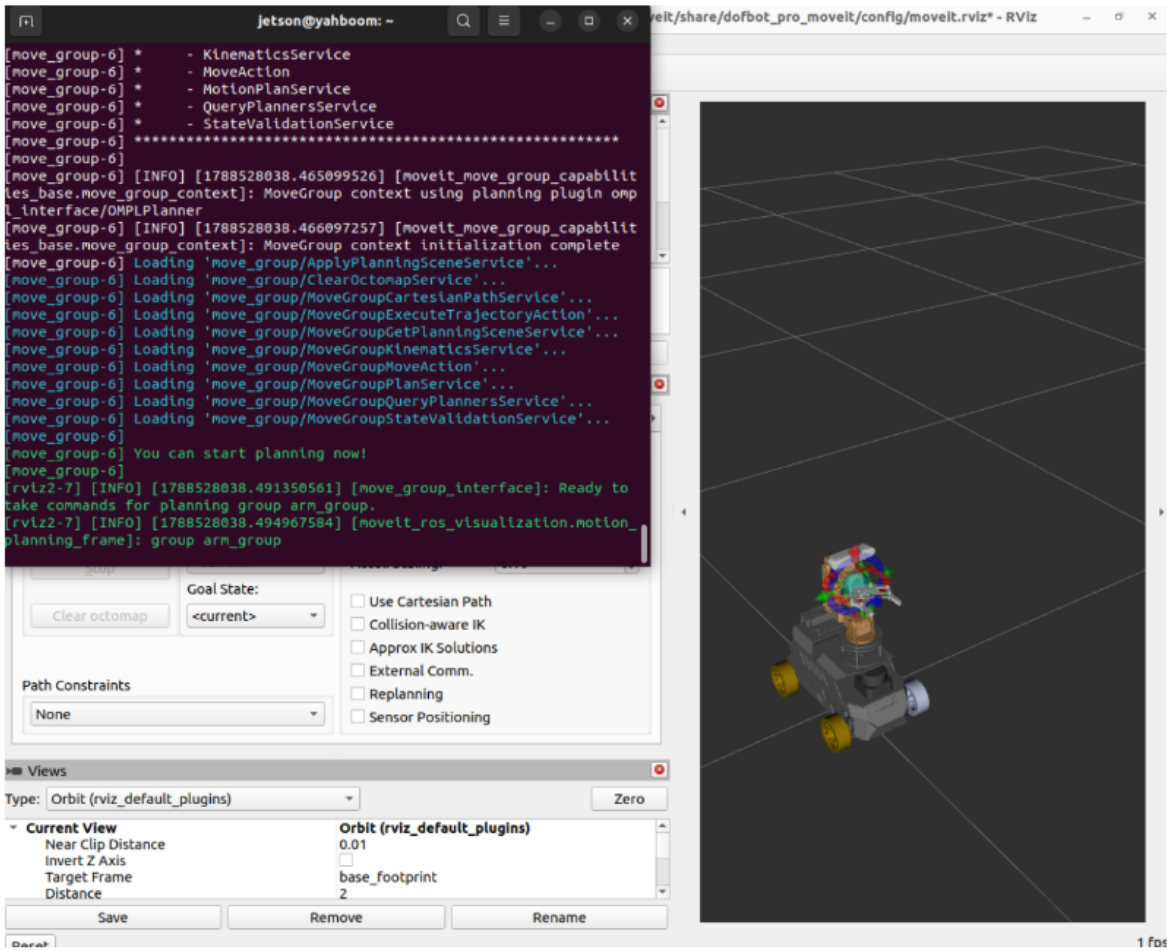
2. Launch and Operation Steps

2.1 Launching the MoveIt2 Simulation Environment

Enter the following command in the terminal to launch the MoveIt2 simulation environment:

```
ros2 launch dofbot_pro_moveit demo_3d.launch.py
```

Once the program starts, when **You can start planning now!** is output in the terminal, it means the MoveIt2 collision detection and planning services are ready, as shown in the figure below:



2.2 Starting Real-Machine Linkage (Optional)

If you want the simulated obstacle-avoidance motion of the robot arm to synchronously control the real physical robot arm, open another terminal and run the real hardware driver node:

```
ros2 run moveIt_demo SimulationToMachine
```

(It is recommended not to start this node during the initial test. First verify that the obstacle-avoidance trajectory is correct in the pure RViz simulation environment before connecting to the real machine).

2.3 Running the Obstacle Avoidance Planning Program

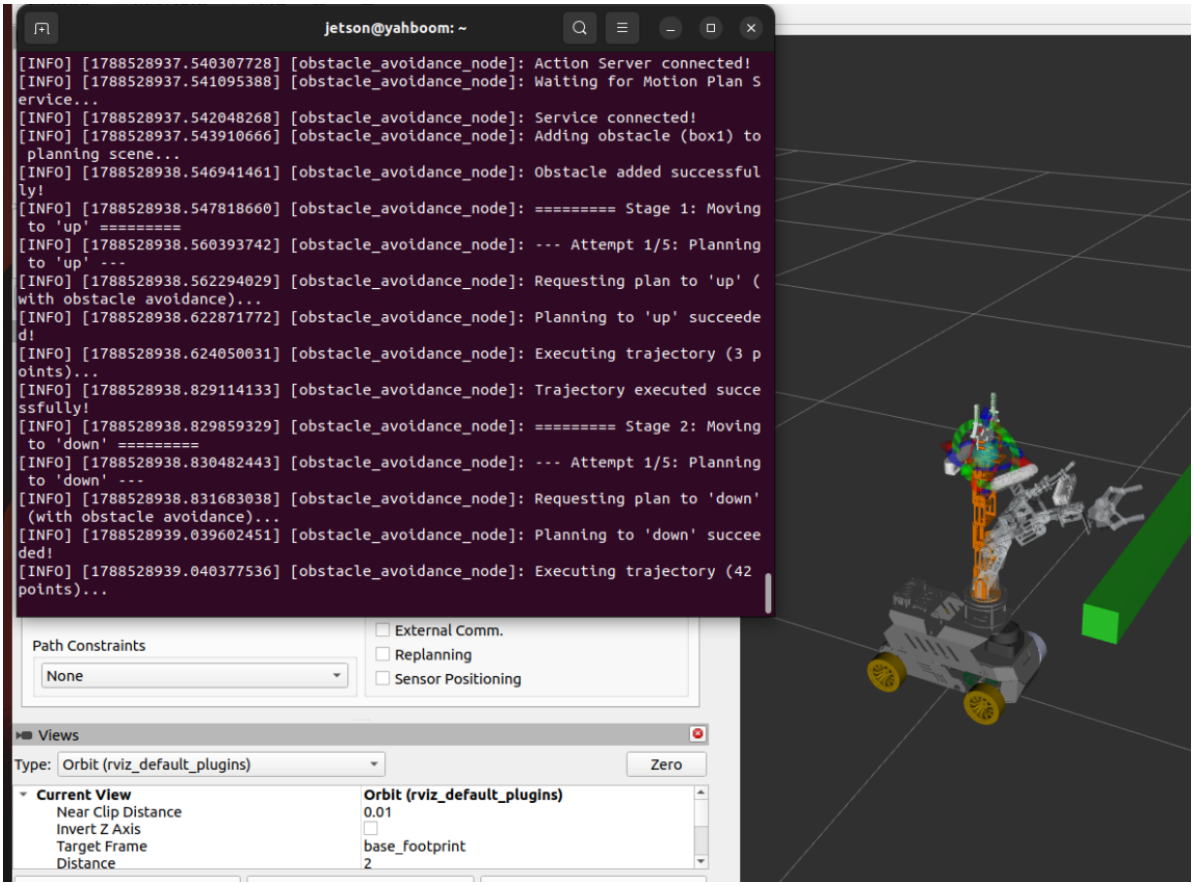
Open a new terminal and enter the following command to run the collision detection and obstacle avoidance motion program:

```
ros2 run moveIt_demo obstacle_avoidance
```

After the program runs:

1. The program automatically publishes a cuboid obstacle (`box1`) into the planning scene in RViz, and a pillar obstacle appears directly in front of the robot arm in RViz;
2. The robot arm first moves to the initial upright pose (`up`);
3. Next, MoveIt starts to plan the motion path from the `up` pose to the head-down pose (`down`). Since the straight path directly ahead is blocked by the cube obstacle, MoveIt's planning algorithm automatically plans an arc that curves upward to go around the obstacle, and drives the robot arm to safely avoid the obstacle and reach the `down` pose;

4. Finally, the robot arm plans again to return from the `down` pose to the `up` pose, likewise automatically going around the obstacle and returning to the origin. As shown in the figure below:



3. Core Code Analysis

Program source code path:
`~/yahboomcar_ros2_ws/dofbot_pro_ws/src/moveIt_demo/moveIt_demo/obstacle_avoidance.py`

3.1 Node Initialization and Core Interfaces

The program inherits from `rclpy.node.Node` and defines three core communication interfaces:

1. `_planning_scene_pub`: Publishes planning scene diff update messages (`PlanningScene`) to the `/planning_scene` topic, used to dynamically add or remove obstacles in the world;
2. `_plan_client`: The MoveIt path planning service client, calling `/plan_kinematic_path` (service type `GetMotionPlan`), where the MoveIt kernel performs collision-checked trajectory search;
3. `_action_client`: The Action client, responsible for sending the planned collision-free trajectory to `/arm_group_controller/follow_joint_trajectory`.

```
import time
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory
from geometry_msgs.msg import Pose
from moveit_msgs.msg import (
    PlanningScene,
```

```
    CollisionObject,
    Constraints,
    JointConstraint,
)
from moveit_msgs.srv import GetMotionPlan
from shape_msgs.msg import SolidPrimitive

class ObstacleAvoidance(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance_node')
        self.get_logger().info("Initializing ObstacleAvoidance node...")

        # 1. Trajectory execution Action client
        self._action_client = ActionClient(
            self,
            FollowJointTrajectory,
            '/arm_group_controller/follow_joint_trajectory'
        )

        # 2. Planning scene publisher (injects obstacles into the environment)
        self._planning_scene_pub = self.create_publisher(PlanningScene,
'/planning_scene', 10)

        # 3. Motion planning service client (planner with collision detection)
        self._plan_client = self.create_client(GetMotionPlan,
'/plan_kinematic_path')

        # 4. The 5 rotating joint names of the robot arm
        self.joint_names = [
            "arm1_Joint", "arm2_Joint", "arm3_Joint",
            "arm4_Joint", "arm5_Joint"
        ]

        # 5. Preset target named states (in radians)
        self.named_states = {
            "up":   [0.0,  0.0,     0.0,  0.0, 0.0],
            "down": [0.0, -1.5708,  0.0,  0.0, 0.0],
        }
```

3.2 Dynamically Adding a Collision Obstacle (add_collision_box)

Construct a `CollisionObject` message to add a 3D geometric primitive to the planning scene:

- `primitive.type = SolidPrimitive.BOX`: The obstacle type is a cuboid;
- `primitive.dimensions = [0.05, 0.05, 0.5]`: The length, width, and height are 5cm, 5cm, and 50cm respectively;
- `box_pose`: Sets the coordinates of the pillar center in the base frame `base_link` to $(x=0.35, y=0.0, z=0.35)$;
- `collision_object.operation = CollisionObject.ADD`: The operation type is adding a new obstacle.

```
    def add_collision_box(self):
        """Add a 3D obstacle (box1) to the MoveIt planning scene"""
        scene = PlanningScene()
        scene.is_diff = True  # Update the scene incrementally
```

```
        collision_object = CollisionObject()
        collision_object.header.frame_id = "base_link"
        collision_object.header.stamp = self.get_clock().now().to_msg()
        collision_object.id = "box1"

        # Cuboid dimensions: 0.05m x 0.05m x 0.5m
        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.05, 0.05, 0.5]

        # Spatial pose of the obstacle
        box_pose = Pose()
        box_pose.orientation.w = 0.7071
        box_pose.orientation.x = 0.7071
        box_pose.position.x = 0.35
        box_pose.position.y = 0.0
        box_pose.position.z = 0.35

        collision_object.primitives.append(primitive)
        collision_object.primitive_poses.append(box_pose)
        collision_object.operation = CollisionObject.ADD

        scene.world.collision_objects.append(collision_object)

        # Publish the scene update
        self._planning_scene_pub.publish(scene)
        time.sleep(1.0)  # Ensure the scene is fully synchronized to the MoveIt
planning pipeline
```

3.3 Trajectory Planning with Obstacle Avoidance Constraints (plan_to_named_target)

Construct a `GetMotionPlan.Request`, set the planning group to `arm_group`, set the start state to the current positions of each joint, and construct the target state as `JointConstraint`s containing the tolerance range of each joint.

Set `num_planning_attempts = 10` and `allowed_planning_time = 5.0`. MoveIt will automatically invoke the OMPL sampling planner to compute a globally optimal motion path while satisfying collision-free conditions:

```
    def plan_to_named_target(self, target_name, current_joints):
        """Request obstacle-avoidance trajectory planning from MoveIt"""
        target_joints = self.named_states[target_name]

        request = GetMotionPlan.Request()
        req = request.motion_plan_request
        req.group_name = "arm_group"

        # Set the start state
        req.start_state.joint_state.name = self.joint_names
        req.start_state.joint_state.position = list(current_joints)

        # Build the target joint constraints
        constraints = Constraints()
        for i, name in enumerate(self.joint_names):
            jc = JointConstraint()
```

```
            jc.joint_name = name
            jc.position = target_joints[i]
            jc.tolerance_above = 0.01
            jc.tolerance_below = 0.01
            jc.weight = 1.0
            constraints.joint_constraints.append(jc)
        req.goal_constraints.append(constraints)

        req.num_planning_attempts = 10  # Maximum number of planning attempts
        req.allowed_planning_time = 5.0  # Maximum allowed computation time
(seconds)

        future = self._plan_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        response = future.result()

        if response.motion_plan_response.error_code.val == 1:  # SUCCESS
            self.get_logger().info(f"Obstacle-avoidance trajectory planning for
target '{target_name}' succeeded!")
            return response.motion_plan_response.trajectory
        else:
            self.get_logger().error(f"Planning failed, error code:
{response.motion_plan_response.error_code.val}")
            return None
```

3.4 Full Task Flow Execution (run Method)

In the main flow, first inject the obstacle into the environment, then perform multiple round-trip obstacle-avoidance planning and tracking executions between the `up` and `down` states. If planning fails, a built-in retry mechanism takes over:

```
    def run(self):
        # 1. Inject the obstacle
        self.add_collision_box()

        current_joints = self.named_states["up"]

        # 2. Stage 1: Move to the upright state (up)
        success, current_joints = self.plan_and_execute("up", current_joints)
        if not success: return

        # 3. Stage 2: Move around the obstacle to the head-down state (down)
        success, current_joints = self.plan_and_execute("down", current_joints)
        if not success: return

        # 4. Stage 3: Avoid the obstacle again and return to the upright state
(up)
        success, current_joints = self.plan_and_execute("up", current_joints)
        if not success: return

        self.get_logger().info("All collision detection and obstacle avoidance
motions completed successfully!")
```